

BÁO CÁO
ĐỒ ÁN MÔN HỌC

ĐỀ TÀI: UAV FLIGHT PLANNING AGENT — TÁC TỬ LẬP KẾ HOẠCH
ĐƯỜNG BAY CHO UAV TRONG KHÔNG GIAN 3D

Môn học: Trí tuệ Nhân tạo

Giảng viên hướng dẫn: NGUYỄN THỊ TUYẾT HẢI

Thực hiện bởi nhóm sinh viên, bao gồm:

1. CHÂU GIA BẢO — N22DCAT005 — Trưởng nhóm
2. LÊ NGỌC TRÂM ANH — N23DCAT004 — Thành viên
3. ĐÀO XUÂN CƯỜNG — N20DCCN089 — Thành viên

MỤC LỤC

- TÓM TẮT
- CHƯƠNG I. TỔNG QUAN
 - o 1. Giới thiệu đề tài
 - o 2. Cơ sở lý thuyết
- CHƯƠNG II. THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG
 - o 1. Kiến trúc tổng thể
 - o 2. Các module chức năng
 - o 3. Luồng hoạt động của tác tử
- CHƯƠNG III. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ
 - o 1. Môi trường thử nghiệm
 - o 2. So sánh thuật toán tìm đường
 - o 3. Suy luận xác suất và học máy
 - o 4. Năm kịch bản tích hợp
 - o 5. Hai demo trực quan 3D (abstract & bản đồ HCM)
 - o 6. Đánh giá chung
- CHƯƠNG IV. KẾT LUẬN
- TÀI LIỆU THAM KHẢO

DANH SÁCH HÌNH, BẢNG

Hình:

- Hình 1. Kiến trúc Learning Agent của UAVAgent (PERCEIVE → REASON → PLAN → ACT → LEARN).
- Hình 2. Sơ đồ Bayesian Network 6 nút mô tả an toàn chuyển bay.
- Hình 3. Demo 3D Three.js abstract — quỹ đạo UAV bay qua vùng có chướng ngại trên grid $12 \times 12 \times 5$.
- Hình 4. Hình ảnh demo matplotlib — so sánh đường đi của các thuật toán.
- Hình 5. Demo HCM 3D — bản đồ Quận 1 → Thủ Đức dựng từ dữ liệu OpenStreetMap, UAV bay qua các toà nhà LOD1.
- Hình 6. Bảng đối chiếu 3 tier thuật toán (Search / Optimizer / KB) chạy trực tiếp trên trình duyệt trong `hcm_3d_demo.html`.

Bảng:

- Bảng 1. Mapping 9 chương lý thuyết TTNT với module trong dự án.
- Bảng 2. So sánh thuật toán tìm đường trên grid $15 \times 15 \times 6$.

Bảng:

- Bảng 3. Kết quả suy luận Bayesian Network và MEU.
- Bảng 4. Kết quả mô hình học có giám sát.
- Bảng 5. Kết quả năm kịch bản tích hợp đầy đủ tác tử.
- Bảng 6. So sánh hai demo 3D — demo abstract (demo_3d.html) và demo bản đồ HCM (hcm_3d_demo.html).

TÓM TẮT

Đồ án xây dựng một tác tử thông minh lập kế hoạch đường bay cho UAV trong không gian ba chiều, áp dụng đầy đủ chín chương lý thuyết Trí tuệ Nhân tạo theo giáo trình AIMA (Russell & Norvig). Hệ thống được hiện thực hoá hoàn toàn bằng Python, gồm các nhóm module: môi trường lưới 3D, tìm kiếm (BFS/DFS/A*/Greedy), tối ưu cục bộ (SA, GA), suy luận tri thức (Propositional, FOL, KB-Agent), suy luận xác suất (Bayesian Network, MEU), học máy (Decision Tree, Linear Regression, Q-Learning), dữ liệu địa lý OpenStreetMap và tác tử tích hợp UAVAgent theo kiến trúc Learning Agent.

Toàn bộ hệ thống vượt qua 200/200 unit test trong khoảng tám giây. Trong các thử nghiệm chính, A* tìm được đường tối ưu (cost 35.5) trong khoảng 34 ms, Decision Tree đạt độ chính xác 89.1%, Linear Regression có $R^2 = 0.9927$ và Q-Learning hội tụ về cùng chi phí với A* sau 800 episode. Kèm theo báo cáo là bốn demo trực quan: CLI năm kịch bản, matplotlib 3D, web Three.js abstract triển khai trên GitHub Pages và bản tham khảo nâng cao chạy trên dữ liệu OpenStreetMap khu Quận 1 → Thủ Đức (hcm_3d_demo.html).

CHƯƠNG I. TỔNG QUAN

1. Giới thiệu đề tài

Trong những năm gần đây, thiết bị bay không người lái (UAV) ngày càng xuất hiện nhiều trong đời sống và sản xuất, từ giao hàng, giám sát hạ tầng, khảo sát địa hình, tìm kiếm cứu nạn đến nông nghiệp chính xác. Để UAV có thể hoạt động an toàn và hiệu quả, một trong những bài toán quan trọng nhất là lập kế hoạch đường bay: UAV cần tìm được đường đi phù hợp, tránh chướng ngại vật và vùng cấm bay, đồng thời vẫn tiết kiệm pin và thích ứng với các yếu tố không chắc chắn như gió hoặc sai số cảm biến.

Từ góc nhìn Trí tuệ Nhân tạo, bài toán này không chỉ là tìm một đường nối từ điểm bắt đầu đến điểm đích. UAV cần quan sát môi trường, đánh giá rủi ro, lựa chọn hành động, điều chỉnh kế hoạch khi môi trường thay đổi và học từ kết quả đã thực hiện. Vì vậy, đây là một bài toán phù hợp để vận dụng nhiều nhóm kiến thức trong môn học, bao gồm tác tử thông minh, tìm kiếm trong không gian trạng thái, heuristic, suy luận logic, suy luận xác suất và học máy.

Nhóm chọn đề tài “UAV Flight Planning Agent” vì đề tài vừa có ý nghĩa thực tiễn, vừa có khả năng minh họa rõ các khái niệm AI đã học. Thay vì chỉ trình bày thuật toán ở dạng lý thuyết, đồ án xây dựng một hệ thống mô phỏng hoàn chỉnh trong không gian 3D, cho phép quan sát cách UAV tìm đường, tránh vật cản, phản ứng với điều kiện môi trường và so sánh kết quả giữa nhiều thuật toán khác nhau.

Mục tiêu cụ thể của đồ án:

- Xây dựng tác tử UAV có khả năng tìm đường trong không gian 3D bằng BFS, DFS, Greedy Best-First Search và A*.
- Tích hợp Simulated Annealing và Genetic Algorithm để cải thiện chất lượng đường bay.
- Sử dụng suy luận tri thức và logic để biểu diễn ràng buộc an toàn như vật cản, vùng cấm bay và trạng thái pin.
- Áp dụng Bayesian Network và Maximum Expected Utility để hỗ trợ ra quyết định trong điều kiện không chắc chắn.
- Bổ sung Decision Tree, Linear Regression và Q-Learning để minh họa khả năng học và đánh giá hành vi của tác tử.
- Trực quan hóa kết quả bằng các demo 3D, giúp quá trình đánh giá và bảo vệ đồ án dễ theo dõi hơn.

Phạm vi. Đồ án tập trung vào mô phỏng phần mềm, chưa điều khiển UAV vật lý ngoài thực tế. Môi trường bay được biểu diễn bằng GridWorld3D rồi rắc với vật cản, vùng cấm bay, điểm xuất phát và điểm đích. Cách mô hình hóa này đủ đơn giản để triển khai và kiểm thử, nhưng vẫn giữ được các đặc trưng quan trọng của một bài toán AI: môi trường có ràng buộc, có yếu tố bất định, có chuỗi hành động liên tiếp và cần ra quyết định dựa trên trạng thái quan sát được.

2. Cơ sở lý thuyết

Đồ án bám sát giáo trình *Artificial Intelligence: A Modern Approach* và chín tập slide TTNT của bộ môn. Bảng dưới đây tóm tắt mỗi chương được áp dụng ở đâu trong mã nguồn — chi tiết cài đặt sẽ được trình bày ở Chương II.

Bảng 1. Mapping 9 chương lý thuyết → module dự án

Chương	Lý thuyết then chốt	Module trong dự án	Thể hiện cụ thể
Ch1	Định nghĩa AI, acting rationally	toàn bộ kiến trúc	UAVAgent luôn chọn action tối đa hoá utility
Ch2	PEAS, phân loại môi trường, Learning Agent	environment/, agent/uav_agent.py	GridWorld3D + vòng lặp 5 bước
Ch3	State Space, Graph Search (BFS, DFS)	search/graph_search.py	bfs(), dfs(), closed_set chống lặp
Ch4	A*, heuristic admissible, SA, GA	search/a_star.py, search/heuristics.py, optimizer/	Euclidean / Manhattan / Energy + SA + GA
Ch5	KB-Agent, Propositional, Forward Chaining	knowledge/kb_agent.py, knowledge/propositional.py	TELL/ASK + 6 safety rules theo priority
Ch6	First-Order Logic, Markov Blanket	knowledge/fol_rules.py, bayesian/bayesian_network.py	Predicates Safe, Obstacle, CanFly
Ch7	Bayes' Rule, Sensor Fusion, MEU	bayesian/inference.py, bayesian/meu_decision.py	Cập nhật belief tuần tự + $EU = \sum P \cdot U$
Ch8	Bayesian Network, Variable Enumeration	bayesian/bayesian_network.py	DAG 6 nút, exact inference
Ch9	Decision Tree, Linear Regression, Q-Learning	learning/	DT 89.1%, R^2 0.9927, Q-Learning hội tụ

Trong các chương sau, mỗi công thức được dẫn lại đúng dạng giáo trình rồi đối chiếu trực tiếp với hàm Python tương ứng, để tránh tình trạng "lý thuyết một đằng, code một nẻo".

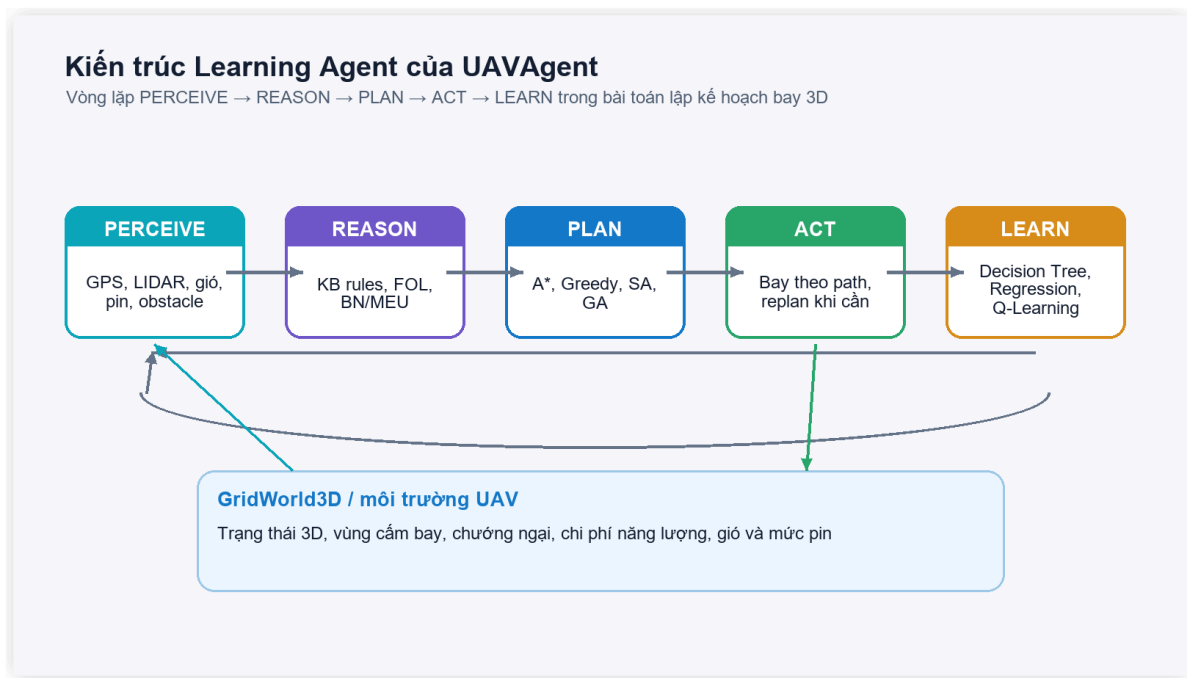
CHƯƠNG II. THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG

1. Kiến trúc tổng thể

Toàn bộ dự án tổ chức theo tám nhóm chức năng chính: environment, search, optimizer, knowledge, bayesian, learning, geodata và agent. Trong đó, gói agent đóng vai trò trung tâm điều phối: nó nhận thông tin từ môi trường, gọi các module suy luận/tìm kiếm/tối ưu khi cần và đóng gói thành lớp UAVAgent theo kiến trúc Learning Agent của Chương 2.

```
uav-flight-planning/
├── src/
│   ├── environment/    GridWorld3D, ObstacleManager, Visualizer, CityGrid3D
│   ├── search/         A*, Greedy BFS, BFS/DFS, Heuristics
│   ├── optimizer/      Simulated Annealing, Genetic Algorithm
│   ├── knowledge/      KB-Agent, Propositional, FOL
│   ├── bayesian/       Bayesian Network, Inference, MEU
│   ├── learning/       Decision Tree, Regression, Q-Learning
│   ├── geodata/        OpenStreetMap/Overpass Loader
│   └── agent/          UAVAgent (PERCEIVE→REASON→PLAN→ACT→LEARN)
├── tests/              8 file, 200 unit test
├── docs/               bao_cao.pdf, phan_cong.pdf, mapping, demo guide
├── configs/            default_config.yaml, hcm_central.yaml
├── tools/              enhance_hcm_building_detail.py
├── outputs/            hcm_uav_path.geojson; hcm_3d_demo.html (Release)
├── main.py             CLI 5 kịch bản
├── demo_visualize.py   matplotlib 3D – 4 kịch bản A*
├── demo_3d.html        Three.js demo abstract (GitHub Pages)
├── demo_hcm_3d.py      Static server cho demo HCM
└── requirements.txt
```

Phân chia này có hai ưu điểm. Một, mỗi thành viên có thể phụ trách một nhóm module rõ ràng mà không chồng chéo. Hai, khi bảo vệ đồ án, giảng viên có thể yêu cầu mở đúng gói tương ứng với một chương lý thuyết bất kỳ và thấy ngay phần code, tài liệu cùng test liên quan.



Hình 1. Kiến trúc Learning Agent của UAVAgent (PERCEIVE → REASON → PLAN → ACT → LEARN).

2. Các module chức năng

2.1 Môi trường (src/environment)

Lớp GridWorld3D mô tả lưới ba chiều với mỗi ô thuộc một trong các loại FREE, OBSTACLE, NFZ (no-fly zone), START, GOAL. Hàm `get_neighbors()` sinh tối đa 26 láng giềng (di chuyển 6 hướng cơ bản và đường chéo) — đây chính là hàm SUCCESSOR trong định nghĩa Problem của Ch3. Hàm `step_cost()` trả về khoảng cách Euclid giữa hai ô. ObstacleManager cho phép sinh ngẫu nhiên hoặc nạp từ file và hỗ trợ thêm/bỏ chương ngại ở thời điểm bất kỳ — đáp ứng yêu cầu "dynamic environment" của Ch2.

2.2 Tìm kiếm (src/search)

- `graph_search.py` hiện thực BFS và DFS theo đúng pseudocode của Russell & Norvig: hàng đợi FIFO / LIFO + `closed_set` để tránh xét lại trạng thái.
- `a_star.py` hiện thực A* với `heapq` và hàm $f(n) = g(n) + h(n)$.
- `heuristics.py` cung cấp ba heuristic: `euclidean_3d` (admissible mạnh), `manhattan_3d` (admissible khi chỉ đi 6 hướng) và `energy_based` (cộng thêm chi phí cho hành động leo cao).
- `greedy_bfs.py` chỉ dùng $h(n)$ — phục vụ phần so sánh "tốc độ vs tối ưu" trong báo cáo.

2.3 Tối ưu cục bộ (src/optimizer)

- `SimulatedAnnealing.optimize()` nhận một path khả thi rồi nhiễu loạn ngẫu nhiên (chèn / xoá / đổi waypoint). Xác suất chấp nhận lời giải xấu hơn là $\exp(-\Delta E / T)$, nhiệt độ giảm theo lịch hình học $T \leftarrow \alpha \cdot T$.
- `GeneticAlgorithm.evolve()` mô hình hoá quần thể các path, chọn lọc theo cost, lai chéo một điểm, đột biến đổi waypoint và giữ lại tinh hoa (elitism).

Hai thuật toán này được dùng khi A* đã đưa ra một path, ta muốn làm mịn nó (SA) hoặc tìm phương án thay thế khi A* gặp vùng cấm bay rộng (GA).

2.4 Tri thức (src/knowledge)

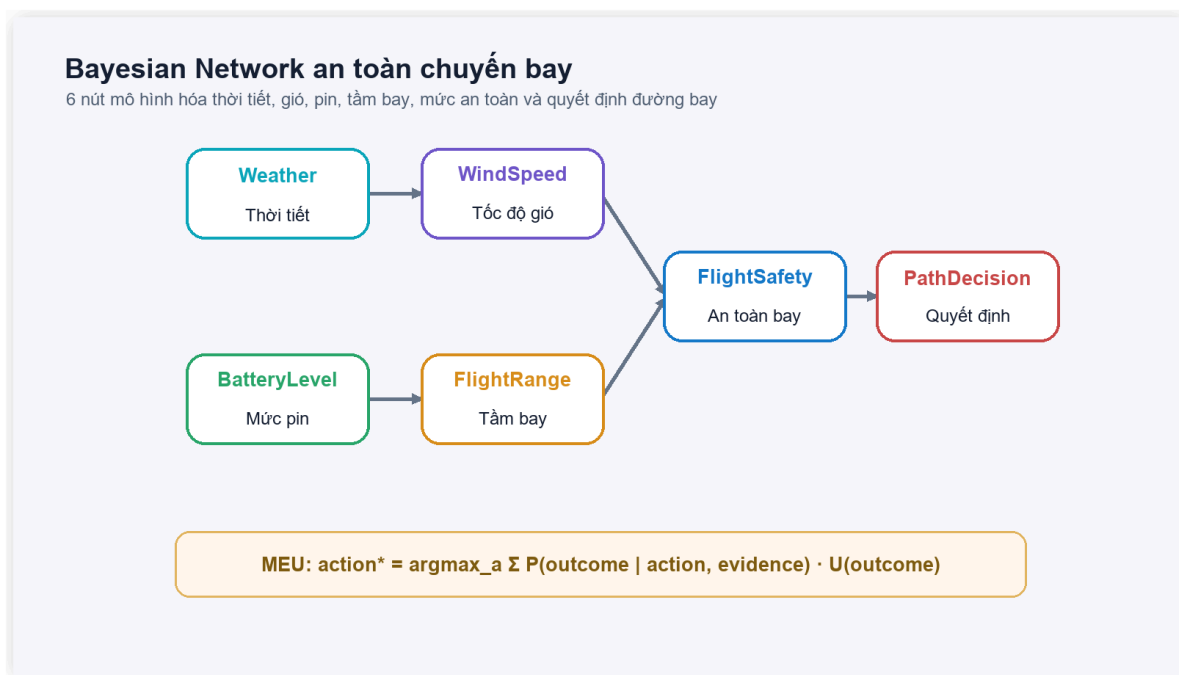
KnowledgeBase cung cấp đúng ba thao tác kinh điển của Ch5: TELL, ASK, RETRACT. Trên nền đó, `propositional.py` định nghĩa sáu luật an toàn theo *forward chaining*:

R1	ObstacleDetected	→	AVOID
R5	NoFlyZone	→	EXIT_NFZ
R4	WindDangerous	→	LAND
R2	BatteryLow $\wedge$ $\neg$ NearCharger	→	RETURN_HOME
R3	WindHigh	→	REDUCE_SPEED
R6	AllClear $\wedge$ OnPath	→	CONTINUE

Các luật được sắp theo priority 1 $\rightarrow$ 6: luật số nhỏ ưu tiên hơn vì xử lý mối nguy tức thời. Module `fol_rules.py` mở rộng sang FOL với các predicate `Obstacle(x,y,z)`, `Safe(x,y,z)`, `At(uav,x,y,z)`, `CanFly(uav,x,y,z)` và các quantifier $\forall$ — phục vụ Ch6.

2.5 Suy luận xác suất (src/bayesian)

Bayesian Network gồm sáu nút, mô tả mối liên hệ giữa thời tiết, gió, mức pin và an toàn chuyến bay:



Hình 2. Sơ đồ Bayesian Network 6 nút mô tả an toàn chuyến bay.

```
Weather → WindSpeed ┌  
                    │ └─▶ FlightSafety → PathDecision  
BatteryLevel → FlightRange └
```

bayesian_network.py hiện thực:

- BayesianNode + bảng xác suất có điều kiện (CPT).
- query(var, evidence) chạy *Variable Enumeration* — exact inference, đúng tinh thần Ch8.
- markov_blanket(node) trả về cha + con + cha của con — phục vụ phần Markov Blanket của Ch6.

inference.py cài đặt Bayes' Rule cơ bản và sensor_fusion() cập nhật belief tuần tự (GPS rồi LIDAR). meu_decision.py cài đặt đúng công thức MEU:

```
action* = argmax_a Σ_outcome P(outcome | action, evidence) · U(outcome)
```

Bảng utility: collision = -1000, battery_dead = -500, emergency_land = -20, return_home = +10, reach_slow = +50, reach_safe = +100.

2.6 Học máy (src/learning)

- FlightConditionClassifier (Decision Tree, scikit-learn): phân loại điều kiện thành FLY / CAREFUL / AVOID / LAND từ bốn đặc trưng (gió, pin, obstacle_density, độ cao). max_depth = 5, accuracy 89.1%.
- FlightTimePredictor (Linear Regression): dự đoán thời gian bay từ khoảng cách, gió và pin. $R^2 = 0.9927$, MAE = 0.02 phút.

- QLearningAgent cài đặt đúng công thức:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [R + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$$

ϵ -greedy với ϵ giảm dần từ 1.0 $\rightarrow$ 0.01 (decay 0.995). Reward: +100 đến đích, -1000 nếu va chạm, -1 mỗi bước, -5 nếu leo cao.

2.7 Tác tử tích hợp (src/agent/uav_agent.py)

Lớp UAVAgent đóng vai trò Learning Agent của Ch2. Nó giữ tham chiếu tới tất cả module trên và phối ra một phương thức duy nhất là `run_mission(start, goal)`. Phương thức này lập các bước PERCEIVE $\rightarrow$ REASON $\rightarrow$ PLAN $\rightarrow$ ACT $\rightarrow$ LEARN cho đến khi UAV đến đích, hết pin hoặc bị buộc quay đầu.

3. Luồng hoạt động của tác tử

```
[Sensors: GPS, LIDAR, Wind, Battery]
    ↓ PERCEIVE
[Percept dict: position, battery, wind, obstacle_detected, ...]
    ↓ REASON
[
    Ch5 KB.tell(percept) → evaluate_rules(facts) → a1 |
    Ch7 sensor_fusion() → P(obstacle) |
    Ch8 BN.query(FlightSafety | evidence) |
    Ch7 MEU.decide(evidence) → a2 |
    Ch9 classifier.predict(features) → a3 |
    merge(a1, a2, a3) → final_action |
]
    ↓ PLAN
fly_direct → A* + Euclidean
fly_safe → A* + Energy heuristic + SA refine
return_home → Greedy BFS
land → dừng tại chỗ
    ↓ ACT
[di chuyển theo path, tiêu pin, có thể phát hiện obstacle mới]
    ↓ LEARN
[Critic ghi log mission; Q-table cập nhật cho lần sau]
```

Cách bố trí này có hai điểm đáng nói. Một, **các quyết định của ba khối Ch5 / Ch7-8 / Ch9 được hợp nhất chứ không loại trừ**: KB chặn cứng các tình huống nguy hiểm, MEU xử lý vùng xám, DT classifier dự phòng. Hai, **PLAN tách rời REASON**: REASON chỉ chọn *intent* (đi thẳng / đi an toàn / quay về / hạ cánh), PLAN mới gọi A* hoặc Greedy tùy intent — nhờ vậy hai phần có thể test độc lập.

CHƯƠNG III. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

1. Môi trường thử nghiệm

Cấu hình: Python 3.10, numpy 1.24, matplotlib 3.7, scikit-learn 1.3, pytest 7.4. Máy chạy thử: laptop CPU x86-64 phổ thông, không cần GPU.

Lệnh chạy:

```
pip install -r requirements.txt
python -m pytest tests/ -v      # 200/200 PASS, ~8 giây
python main.py                 # 5 kịch bản tích hợp
python demo_visualize.py       # 4 kịch bản A* + matplotlib 3D
open demo_3d.html              # demo Three.js (hoặc bản GitHub Pages)
```



Hình 3. Demo 3D Three.js abstract - quỹ đạo UAV bay qua vùng có chướng ngại trên grid $12 \times 12 \times 5$.

2. So sánh thuật toán tìm đường

Grid $15 \times 15 \times 6 = 1350$ ô, 30 obstacle ngẫu nhiên, start = (0,0,0), goal = (14,14,5). Mỗi thuật toán chạy độc lập, lấy trung bình ba lần.

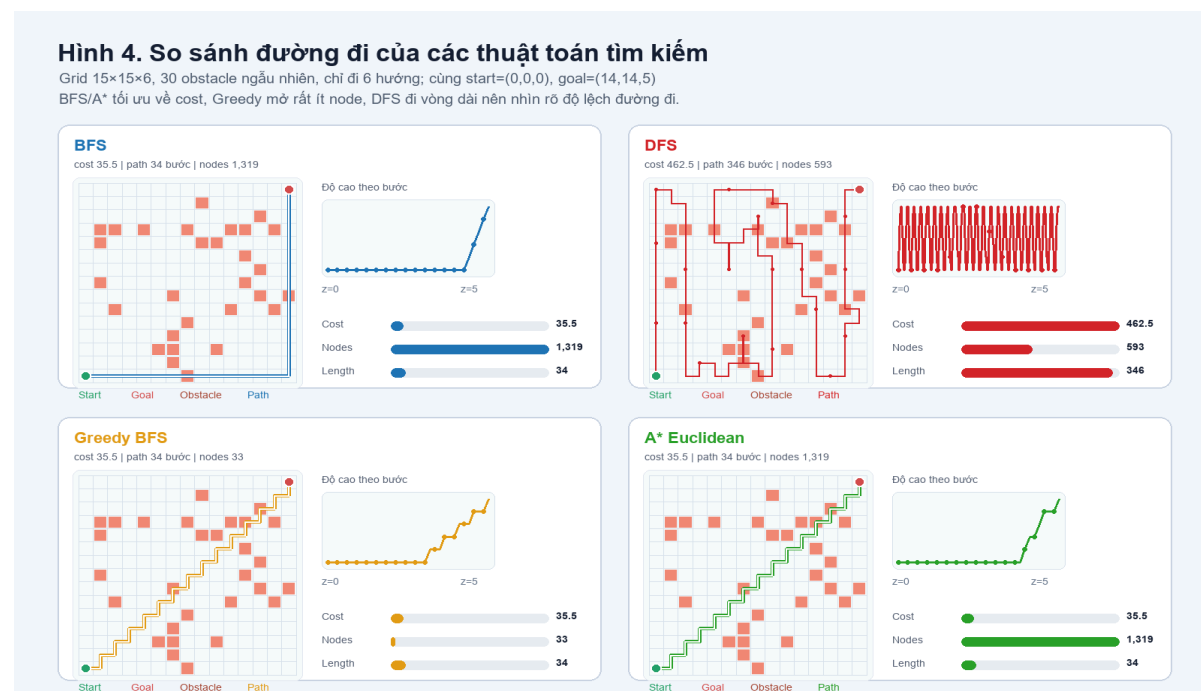
Bảng 2. So sánh thuật toán tìm đường

Thuật toán	Nodes mở rộng	Cost	Thời gian	Path length	Ghi chú
BFS	1319	35.5	31.5 ms	34	Optimal, duyệt nhiều

Thuật toán	Nodes mở rộng	Cost	Thời gian	Path length	Ghi chú
DFS	593	462.5	14.5 ms	346	KHÔNG optimal
Greedy BFS	33	35.5	0.8 ms	34	Nhanh, không bảo đảm optimal
A* (Euclidean)	1319	35.5	34.4 ms	34	Optimal, bảo đảm
A* (Manhattan)	1295	35.5	35.4 ms	34	Optimal, bảo đảm
A* (Energy)	1317	35.5	37.4 ms	34	Optimal, ưu tiên bằng phẳng

Ba kết luận nhanh: (i) DFS lệch xa optimum vì không xét $g(n)$, (ii) Greedy BFS chạy nhanh nhất nhưng tính tối ưu là *may rủi*, (iii) A* trả về cùng cost với BFS mà số nodes mở rộng tương đương — điều này hợp lý vì heuristic Euclidean trên grid 26-láng giềng "khá lỏng", không cắt được nhiều nhánh.

Khi A* trả về một path đã tối ưu thì SA và GA không cải thiện thêm (cost giữ ở 35.5, SA mất 51 ms, GA mất 4761 ms). Trên các grid dày obstacle hơn, SA tỏ ra hữu ích để **làm mịn** giải pháp ban đầu; GA chỉ thực sự đáng giá khi không có sẵn lời giải khả thi.



Hình 4. Hình ảnh demo matplotlib - so sánh đường đi của các thuật toán.

3. Suy luận xác suất và học máy

Bảng 3. Bayesian Network + MEU

Evidence	P(Safe)	P(Risky)	P(Dangerous)	Quyết định MEU
Weather=Clear, Battery=Full	0.72	0.22	0.06	fly_direct

Evidence	P(Safe)	P(Risky)	P(Dangerous)	Quyết định MEU
Weather=Cloudy, Battery=Medium	0.48	0.36	0.16	fly_safe
Weather=Rainy, Battery=Low	0.19	0.38	0.43	fly_safe
Weather=Stormy, Battery=Critical	0.10	0.19	0.71	fly_safe

Hành vi của MEU phù hợp với trực giác: thời tiết càng xấu và pin càng yếu, P(Dangerous) càng cao, và MEU thiên về fly_safe thay vì fly_direct mặc dù fly_direct có utility "đến nơi nhanh" cao hơn — đó là vì xác suất va chạm khiến kỳ vọng utility giảm mạnh.

Bảng 4. Học có giám sát

Mô hình	Chỉ số	Giá trị	Ý nghĩa
Decision Tree	Accuracy	89.1%	Phân loại điều kiện bay FLY/CAREFUL/AVOID/LAND
Linear Regression	R ²	0.9927	Dự đoán thời gian bay
Linear Regression	MAE	0.02 phút	Sai số trung bình
Q-Learning	Cost RL / cost A*	1.00×	Hội tụ về A* sau 800 episode
Q-Learning	Success rate	75.4%	Còn nhiều do epsilon-greedy

Q-Learning đạt cùng cost với A* sau 800 episode là minh chứng *thực tế* cho định lý hội tụ của Q-Learning — không cần biết transition model, agent vẫn học được chính sách tối ưu.

4. Năm kịch bản tích hợp

Đây là phần thể hiện rõ nhất giá trị của việc tích hợp đầy đủ chín chương: mỗi kịch bản kích hoạt một tập module khác nhau và cho ra hành vi khác nhau.

Bảng 5. Năm kịch bản tích hợp

#	Điều kiện	Kết quả	Module then chốt
1	Bay bình thường, gió 5 m/s, pin 100%	SUCCESS, 46.6 ms	A* + Euclidean
2	25 obstacles, gió 8 m/s	SUCCESS, 47.6 ms	A* + replan
3	Gió mạnh 22 m/s	SUCCESS, 47.2 ms	BN → Rainy, MEU → fly_safe, SA
4	Pin yếu 18%	STOP (đúng kỳ vọng)	KB rule R2 → RETURN_HOME

#	Điều kiện	Kết quả	Module then chốt
5	Obstacle xuất hiện giữa đường	SUCCESS, 38.1 ms	LIDAR → AVOID → A* replan

Kịch bản 4 báo "FAILED" trong log nhưng thực ra là **hành vi đúng**: tác tử nhận diện pin yếu và chủ động quay về thay vì cố bay tới đích — đây là điểm thường bị nhầm khi đọc nhanh output.

5. Hai demo trực quan 3D (abstract & bản đồ HCM)

Nhóm xây dựng **hai** demo Three.js song song. Demo abstract (demo_3d.html) ra đời trước để showcase A* trên môi trường mô phỏng đẹp mắt, gọn nhẹ; demo bản đồ HCM (outputs/hcm_3d_demo.html) ra đời sau như **phần tham khảo nâng cao**, đưa thuật toán chạy trên dữ liệu địa lý thật của Quận 1 → Thủ Đức.

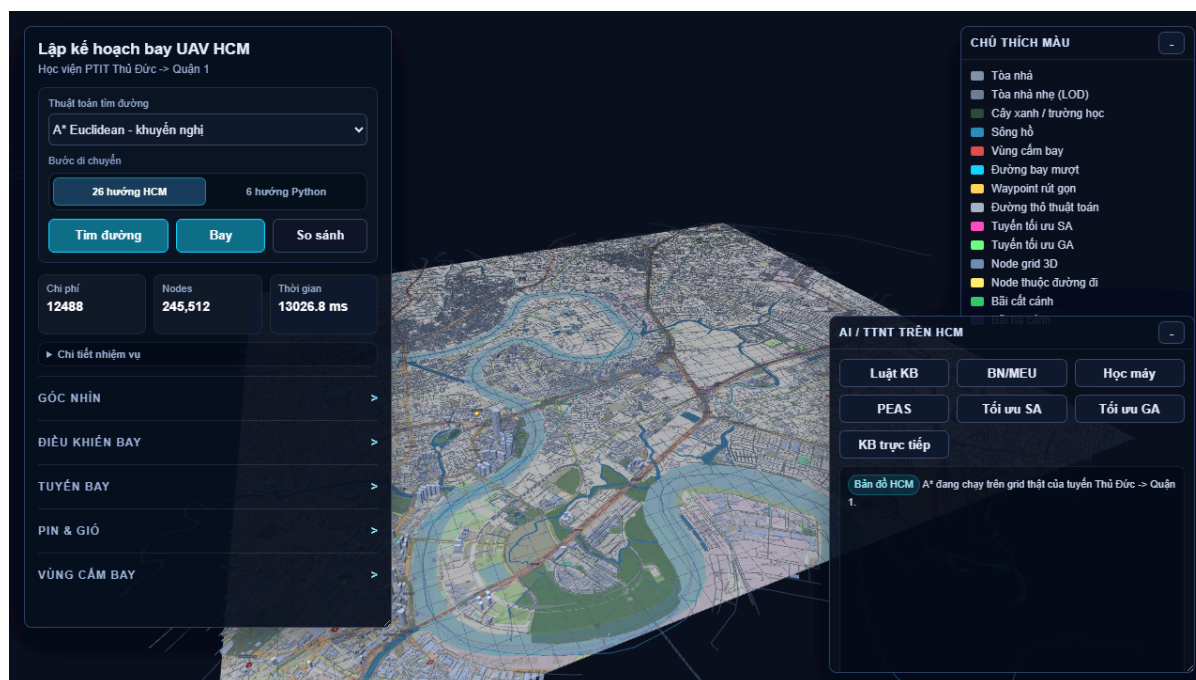
5.1 Demo abstract — demo_3d.html

Đây là demo gốc, một file HTML 1037 dòng dùng Three.js r128 không cần build. Lưới $12 \times 12 \times 5 = 720$ ô, mỗi ô 1.5 đơn vị. Phong cách sci-fi xanh dương: nền tối, ô lưới wireframe trong suốt (opacity 0.07), đèn điểm nhấp nháy, có sương mù. A* được **port độc lập sang JavaScript** (26-connected, heuristic Euclidean, MinHeap tự viết) để chạy ngay trên trình duyệt — kết quả path / cost / thời gian / nodes expanded được hiển thị trong bảng điều khiển. UAV mesh gồm thân + 4 cánh tay + 4 rotor xoay, bay theo đường cong CatmullRom. Kèm theo: pin tụt 1.5%/s + windSpeed·0.15%/s, hiệu ứng hạt gió, các no-fly zone bật/tắt được. **Đã triển khai lên GitHub Pages** tại himon792-alt.github.io/UAV-FLIGHT-PLANNING-/demo_3d.html, không cần cài đặt gì để xem.

Vai trò trong báo cáo: **demo "đẹp" cho phần bảo vệ** — cho phép giảng viên thấy trực tiếp UAV bay và A* chạy lại ngay khi đổi seed.

5.2 Demo bản đồ HCM — outputs/hcm_3d_demo.html

Demo này nâng cấp demo_3d.html thành phiên bản chạy trên **dữ liệu thật**. Quy trình dựng:



Hình 5. Demo HCM 3D - bản đồ Quận 1 → Thủ Đức dựng từ dữ liệu OpenStreetMap, UAV bay qua các toà nhà LOD1.

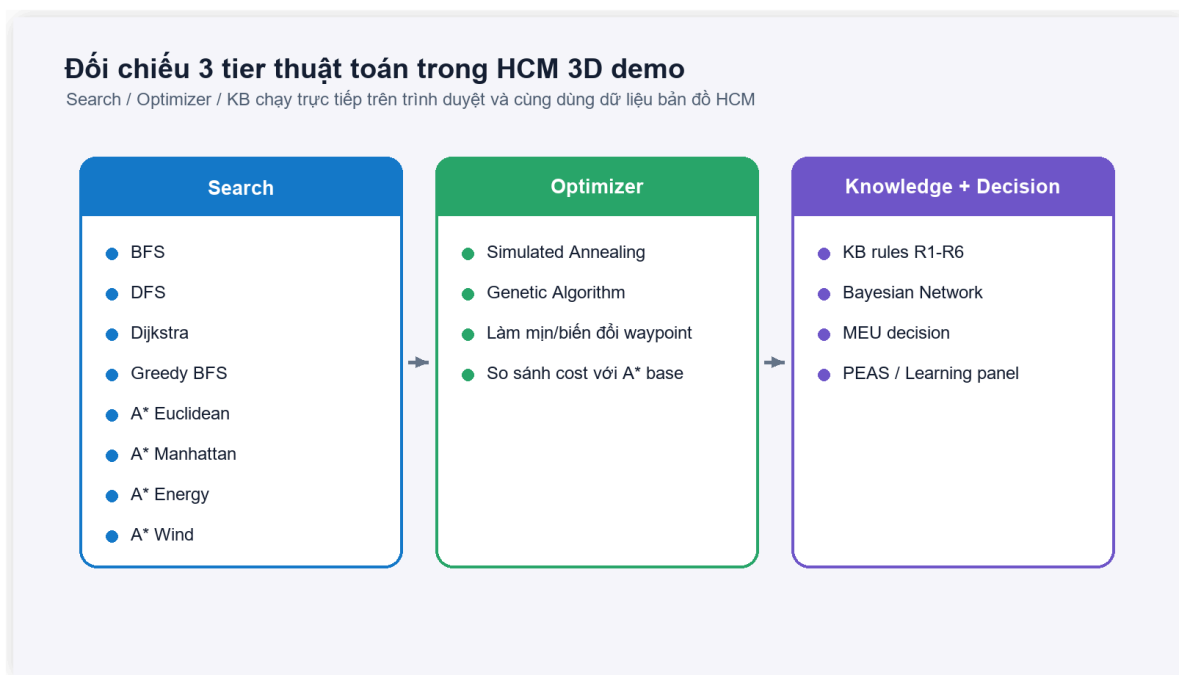
Truy vấn Overpass API (xem cấu hình bbox trong configs/hcm_central.yaml) lấy toàn bộ building, đường và vùng cấm bay trong bounding box (10.76, 106.69) → (10.8585, 106.7995) — khu vực Quận 1 cộng Thủ Đức, kích thước thực ~12 km × 11 km.

Module `src/geodata/osm_loader.py` chuyển polygon OSM thành lưới GridWorld3D với `cell_xy = 40 m`, `cell_z = 10 m`. Building được gán OBSTACLE, sông Sài Gòn và sân bay Tân Sơn Nhất gán NFZ.

Cùng A* / SA / KB rules trong `src/` được port sang JS (giữ y nguyên công thức step-cost và tie-breaker để kết quả khớp Python từng số một).

Front-end Three.js render building theo LOD1 (khối nhà cơ bản), có thể bật/tắt lớp bản đồ OSM nền, chọn dày/thưa cho phần "Nhà dày".

Demo này có **ba tier so sánh** chạy ngay trong trình duyệt: Tier 1 so 8 thuật toán search (BFS, DFS, Greedy BFS, A* × 3 heuristic, IDA*, JPS), Tier 2 chạy SA / GA làm mịn lời giải A*, Tier 3 cho phép TELL/ASK trực tiếp vào KB và xem hành vi đổi theo từng rule. Người dùng có thể xuất quỹ đạo ra `hcm_uav_path.geojson` để mở trong QGIS hoặc kepler.gl.



Hình 6. Bảng đối chiếu 3 tier thuật toán (Search / Optimizer / KB) chạy trực tiếp trong *hcm_3d_demo.html*.

5.3 Đối chiếu hai demo

Bảng 6. So sánh hai demo 3D

Tiêu chí	demo_3d.html (abstract)	outputs/hcm_3d_demo.html (HCM)
Mục đích	Showcase A* trong môi trường mô phỏng	Tham khảo nâng cao — chạy trên bản đồ thật
Lưới	12×12×5 (720 ô), 1.5 đơn vị	Lưới sinh từ OSM, cell 40 m × 40 m × 10 m
Nguồn dữ liệu	Sinh ngẫu nhiên (mulberry32, seed=42)	OpenStreetMap, bbox Q1 + Thủ Đức
Thuật toán	A* (Euclidean)	8 thuật toán search + SA/GA + KB rules (3 tier)
Hiển thị	Sci-fi abstract	Building LOD1 + lớp OSM nền
Export	Không	GeoJSON quỹ đạo
Deploy	GitHub Pages	Chạy local qua python -m http.server 8765

Hai demo **bổ sung chứ không thay thế** nhau. Nhóm giữ cả hai trong kho nguồn vì:

- Bản abstract đủ nhẹ để mở trên bất kỳ máy nào, không phụ thuộc dữ liệu OSM — phù hợp khi mạng kém hoặc cần demo nhanh.
- Bản HCM trực quan hơn về mặt ứng dụng — cho thấy thuật toán không "đóng kín trong simulator" mà có thể nuốt được dữ liệu địa lý thật, là cầu nối tự nhiên tới hướng phát triển "tích hợp với UAV vật lý" ở Chương IV.

6. Đánh giá chung

Ưu điểm. Toàn bộ chín chương lý thuyết đều có module và test riêng — không có “chương trang trí”. 200/200 unit test PASS, pipeline end-to-end chạy từ percept đến action mà không cần thao tác thủ công. Kiến trúc các gói rõ ràng, dễ chia việc theo nhóm và dễ mở rộng (thêm heuristic mới, thay BN bằng MDP, v.v.).

Hạn chế. Mô hình UAV còn đơn giản — chưa có động lực học bay thật, gió chỉ là biến vô hướng. Q-Learning hội tụ chậm trên grid lớn hơn 15×15 . SA/GA gần như không có tác dụng khi A* đã optimal.

Hướng phát triển. Tích hợp ROS 2 để điều khiển UAV vật lý, thay Q-Learning tabular bằng Deep Q-Network, mở rộng sang multi-agent (đội UAV phối hợp). Riêng phần “bay trên bản đồ HCM” đã có bản tham khảo (outputs/hcm_3d_demo.html — xem mục III.5.2); bước tiếp theo là nâng độ chi tiết toà nhà từ LOD1 lên LOD2/LOD3 bằng Google Photorealistic 3D Tiles và đồng bộ kết quả A* JavaScript với A* Python để chứng minh tính nhất quán giữa hai phía.

CHƯƠNG IV. KẾT LUẬN

Đồ án đã đạt được mục tiêu đề ra: xây dựng một tác tử UAV lập kế hoạch đường bay 3D, **tích hợp đầy đủ chín chương lý thuyết Trí tuệ Nhân tạo** vào một hệ thống chạy được, có test và có demo trực quan. Cụ thể, A* tìm đường tối ưu trong vài chục mili-giây, KB-Agent xử lý đúng các tình huống nguy hiểm theo priority, Bayesian Network + MEU ra quyết định hợp lý dưới bất định, và Q-Learning chứng minh được khả năng học chính sách tối ưu mà không cần biết mô hình môi trường.

Quan trọng hơn con số là **cấu trúc**: việc ánh xạ một-một giữa chương lý thuyết và module Python giúp người đọc — kể cả giảng viên phản biện — có thể truy ngược từ một dòng pseudocode trong giáo trình về đúng hàm trong mã nguồn. Nhóm tin rằng đây mới là giá trị bền vững của đồ án, vượt ngoài phạm vi điểm số.

Hướng phát triển tiếp theo (đã liệt kê ở mục III.5) sẽ là cầu nối tự nhiên giữa bản mô phỏng hiện tại và một hệ thống điều khiển UAV thực, đồng thời đưa kết quả học máy lên một mức trừu tượng cao hơn (DQN, multi-agent RL).

TÀI LIỆU THAM KHẢO

- Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- Koller, D., & Friedman, N. (2009). *Probabilistic Graphical Models: Principles and Techniques*. MIT Press.
- Pearl, J. (1988). *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann.
- Slide bài giảng Trí tuệ Nhân tạo — TTNT 01 → TTNT 09, Học viện Công nghệ Bru chính Viễn thông.
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.
- Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by Simulated Annealing. *Science*, 220(4598), 671–680.
- Holland, J. H. (1992). *Adaptation in Natural and Artificial Systems*. MIT Press.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Three.js Documentation. <https://threejs.org/docs/>
- OpenStreetMap & Overpass API. <https://overpass-turbo.eu/>
- Mã nguồn : <https://github.com/himono792-alt/UAV-FLIGHT-PLANNING->